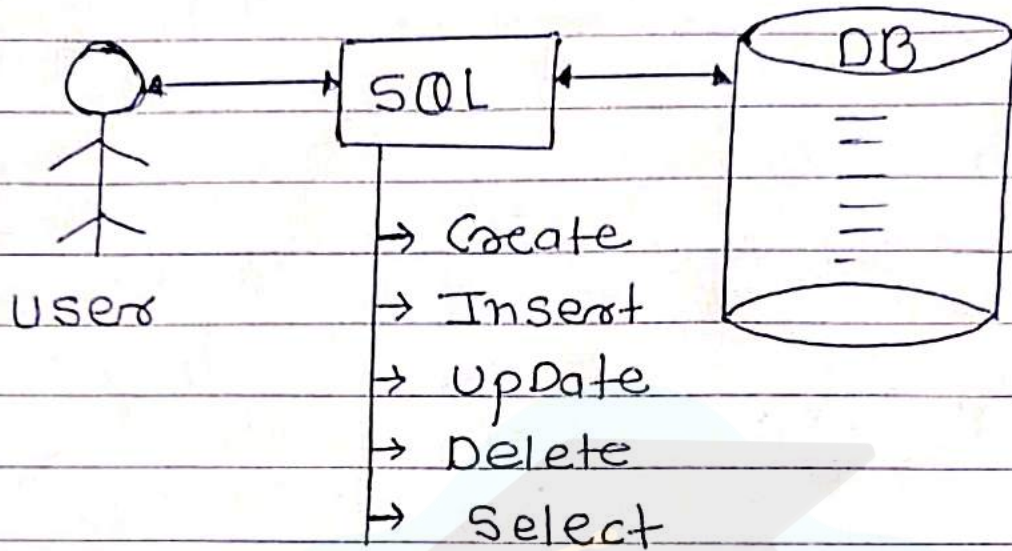
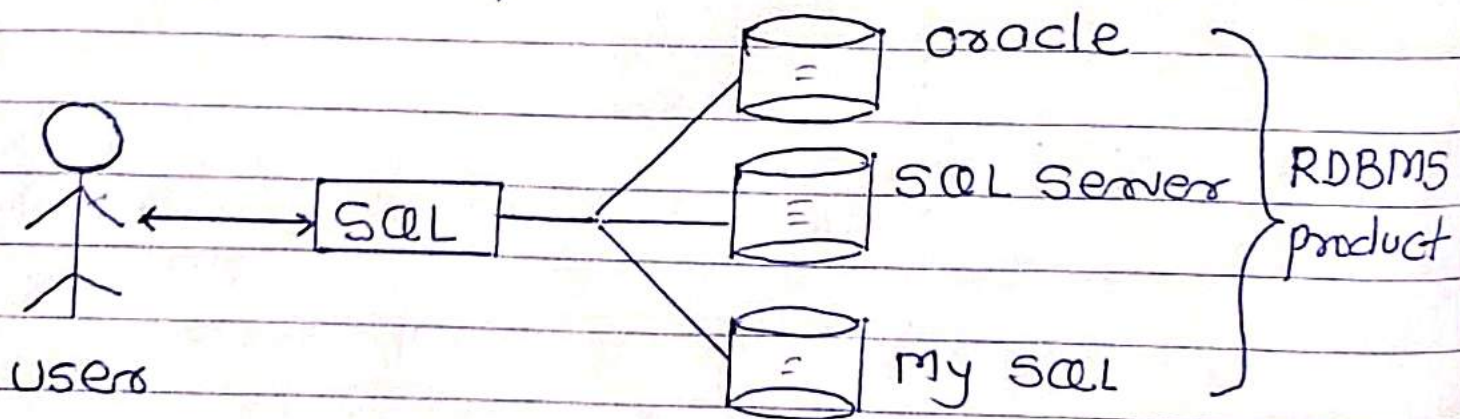


SQL (Structured Query Language)

SQL :- SQL is a non procedural language which was introduced by the IBM in 1970's. Which is used to communicate with database.



- SQL is also called as sequeal or CLI language (Common Language Interface). This is only the language which can use to communicate with any RDBMS product.



SQL is not Case Sensitive language that we can write SQL predefined queries or syntaxes in any case characters. (either upper or lower)

- Every SQL Statement should ends with a semicolon but it is optional in SQL server.

Sublanguages of SQL

① Data Definition Language (DDL)

- Create
- Alter
- Sp_Rename
- Truncate
- Drop

② Data Manipulation Language (DML)

- Insert
- Update
- Delete

③ Data Query Language (DQL)

- Select

④ Transaction Control Language (TCL)

- Commit
- RollBack
- Savepoint

⑤ Data Control Language (DCL)

- Grant
- Revoke

① DDL (Data Definition Language)

This Language Commands are using to define, modify & Drop an object of database from SQL server.

① **Create** :- Creating a new database or new table in SQL Server

Step 1 :- Create a new database in SQL Server

Syntax :- Create database <DB NAME>;

Ex :- Create database MYDB;

Step 2 :- Select The required database from SQL Server.

Syntax :- Use <DB Name>;

Ex :- Use MYDB;

Step 3 :- Create new table in database

Syntax :- Create table <Table Name>

<<Column Name 1> <DT> [Size], <Column Name 2> <DT> [Size] - - - -); -- 1000 Columns

Ex :- Create Table Student (Sid Int, Sname char (10), Sfee Decimal (6, 2), AGE Tinyint);

Step 4 :- To view the structure of table

Syntax :- Sp_HELP <Table Name>;

Ex :- Sp_HELP Student;

sp-HELP is predefined stored procedure.

② **ALTER** :- To change or modify the structure of a table or a database.

by using the Alter Command we can perform a following four operations on existing table.

To perform these operations we required subcommands of alter.

i) Alter - Alter Column

ii) Alter - Add

iii) Sp - Rename

iv) ALTER - Drop

i) Alter - Alter Column :- To change datatype & also size of the datatype of a particular column.

Syntax :-

Alter table <TN> Alter Column <Column Name>
<New DT> [New Size];

Ex :- Alter table Student Alter Column SName
Varchar (50);

ii) Alter - Add :- Adding a new column to add existing table.

Syntax :- Alter table <TN> Add <New Column
Name> <DT> [Size];

Ex :- Alter Table Student Add SAddress
Varchar (30)

iii) Sp-Rename :- To change a Column name or a table name in database.

A) Syntax to change a Column Name in table:

Sp-Rename <TableName>.<OLD Column Name>,
'<New Column Name>';

Ex:- Sp-Rename 'Student.SName', 'StudentNames';

B) Syntax to change a table name in database

Sp-Rename '<Old table Name>', '<New table Name>';

Ex:- Sp-Rename 'student', 'Studentdetails'

OR

Sp-Rename 'Studentdetails', 'Student'

iv) ALTER-Drop :- Dropping a Column from the table

Syntax :- Alter table <Table Name> Drop Column <Column Name>;

Ex:- Alter table Student Drop Column AGE

⑥ Truncate :- Deleting rows from the table but not structure of the table. by using truncate command we can not delete a specific row from the table because it doesn't support 'where' clause condition.

Syntax :- `Truncate table <TableName>;`

Ex :- `Truncate table Student`

⑦ Drop :- Dropping a table from a database permanently

Syntax :- `Drop table <TableName>;`

Ex :- `Drop table Student;`

⑧ DML :- (Data Manipulation Language)

This language commands are used to change or manipulate data in database table

i) Insert :- Inserting a new row into a table. There are two methods to insert rows into a table

i) Implicit Method :- Inserting all values for all columns into a table (without left any column)

Syntax :- `Insert [into] <TableName> values (value1, value2, value3 - - -);`

Ex:- Create table student (STID int, SName varchar (40), Spee decimal (6, 2), Age tinyint)

Ex:-

Insert into student values (101, 'SAI', 2500, 21)

OR

Insert student values (102, 'JAMES', 4500, 23)

ii) Explicit Method :- Inserting value's for required Column's only (with left any Column in the table)

Syntax :-

Insert [INTO] <TableName> (Required Column Names) values (103 'ALLEN')

How to Insert Multiple rows into a table

Syntax for implicit :-

Insert [INTO] <TableName> values (Row1 values), (Row2 values) ... - ?

Ex:-

Insert into student values (104, 'Scott', 1800, 22), (105, 'Hard', 1000, 25)

Syntax for Explicit :-

Insert [INTO] <TableName> (Required Column Names) values ((Row1 values), (Row2 values), ...)

Ex:- Insert student (STID) values (106), (107), (108)

② Update :- Updating all rows data in a table at a time or a specific row data in a table by using 'Where' Condition.

Syntax :-

Update <Table Name> SET <ColumnName1> = <Value1>, <ColumnName2> = <Value2>
---- [Where Condition];

Ex:- Write a query to update employee job as HR, Salary as 14,000 & who's employee Number is 7788

update emp set Job = 'HR', Salary = 14000
where EmpNo = 7788

Ex:- Write a query to update all employee Commission as 500

update Emp set COMM = 500

③ Delete :- Deleting all rows from the table at a time or a specific row from the table by using where clause Condition

Syntax :-

Delete from <TableName> [Where <Condition>];

Ex:- Write a query to delete employee from the table who are working in the job is clerk.

Delete from Emp where job = 'clerk'

Ex:- Write a query to delete all emp details from the table

* Delete from Emp

Difference between delete & Truncate

Delete	Truncate
① It is DML operation	① It is DDL operation
② It can delete a specific row from the table	② It is not possible
③ It support where clause condition	③ It doesn't support where clause condition
④ It is the temporary data deletion	④ It is permanent data deletion
⑤ We can restore deleted data by using rollback	⑤ We can not restore deleted data by using rollback
⑥ Execution speed is slow	⑥ Execution speed is fast.

Note :- In the above query example the employee smith salary is 8000 & there is no Commission so that salary + Comm is 8000 only but it returns Null.

To overcome with above problem we should use a predefined function in SQL server is IS NULL function.

IS Null (exp1, exp2)

- It is a predefined function which is used to replace a user defined values in place of Null.
- This function is having the following two arguments are exp1, exp2
- If expression 1 is Null then it returns expression 2 value (user defined value)
- If exp1 is Not Null then it returns expression 1 value only.

Ex:-
 Select ISNull (Null, 0) AS Result -- 0
 Select ISNull (Null, 100) AS Result -- 100
 Select ISNull (0, 100) AS Result -- 0
 Select ISNull (50, 0) AS Result -- 50

- select EName, Job, Salary, Comm, Salary + ~~Comm~~ (Comm, 0) AS total from ISNULL

Emp Where EName = 'Smith'

	EName	Job	Salary	Comm	Total
1	Smith	clerk	8000	Null	8000

Like :- To perform a database operation (select, update, delete) on specific characters pattern.

- When we work with like operator we should use the following wildcard operator etc.

① % $\Rightarrow$ It represent the remaining group of character's after selected char in the expression.

② _ (underscore) $\rightarrow$ Counting a single char

③ [] $\rightarrow$ It represents set of char.

Syntax :-

Where < Column Name > Like '[< wildcard operator >] < special character > [< wildcard operator >]'

Ex:- To Display employee whose Name starts with 'S' character.

→ Select * from Emp Where EName like 'S%'

Ex:- To display employee whose employee Name is having a second char is 'O'

→ Select * from Emp Where EName like '_O%'

Ex:- To display employee whose Name is having four chars.

→ Select * from Emp Where EName like '____' (No space betⁿ underscore)

Ex:- To display employee whose Name contains 'I' char.

→ Select * from Emp Where EName like '%I%'

Ex:- To display list of employee who are join in the year 1981.

→ Select * from Emp Where Hiredate like '1981%'

(13)

Ex:- To display list of employee who are join in the month of feb.

→ Select * from Emp Where Hiredate like ' % - 02 - % '

* Like operator with special characters

① To Display employee whose name is having @ Symbol

→ Select * from Emp Where EName like ' % . @ % '

② To display employee whose name is having # Symbol

→ Select * from Emp Where EName like ' % # % '

③ To display employee whose Name is having - (underscore) Symbol

→ Select * from Emp Where EName like ' % _ % ' → Wrong Result

Note:- Generally When we use `_ %` symbols in Where condition along with like operator. SQL Server will treat as wildcard operators, but not the special characters. So to avoid this problem we should use the special keyword is "Escape '\'"

Ex:- select * from Empname Where Empname like '% \%' Escape '\'

④ To display employee whose name is having % symbol

→ select * from Empname Where Empname like '% \%' Escape '\'

⑤ To display employee whose name starts with A, C, M, W.

→ select * from Empname Where Empname like '[A, C, M, W] %'

Not Like :-

Ex:- Write a query to display employee details whose name not starts with 'S' character.

→ select * from Empname Where Empname not like 'S %'

* Set Operators :-

Set operators are used to retrieve the data from a single table or multiple tables in vertically.

These operators are :

① Union

② Union all

③ Intersect

④ Except

① Union :- It returns all values from all sets without duplicates.

② Union all :- It returns all values from all sets including duplicates.

③ Intersect :- It returns common values

④ Except :- It returns uncommon values from the left side set but not right side.

* Example on set operators with single table.

Syntax :-

Select * from <TableName> [Where <Condition>]

<Set operator>

Select * from <Table Name> [Where <Condition>]

① Union :-

Ex:- Select job from Emp Where DeptNo=10
 Union
 Select job from Emp Where DeptNo=20

DeptNo 10

Manager

President

clerk

Manager

DeptNo 20

clerk

Manager

Analyst

clerk

clerk

② Union all :-

Ex:- Select Job from Emp Where DeptNo
 =10

Union all

Select Job from Emp Where DeptNo =20

③ Intersect :-

Ex:- Select Job from Emp Where DeptNo=10
 Intersect

Select Job from Emp Where DeptNo =20

★ Example on set operators with multiple tables
 syntax :-

Select * from <TableName> [Where <condition>]

<Set operator>

Select * from <TableName 2> [Where <condition>]

Ex:- Create table Emp-HyD (~~EID~~ int, ~~int~~ #
(Eid int, Ename Varchar(40), Salary ~~int~~ money)

Create table Emp-Chennai (Eid int, Ename
Varchar(40), Salary Money)

① Hyd

	Eid	Ename	Salary
1	101	SAI	85,000
2	102	ADAMS	38000
3	103	JAMES	48000

② Chennai

	Eid	Ename	Salary
1	101	SAI	85,000
2	102	MILLER	62,000
3	103	ALLEN	58,000

* Write a query to display all employee details who are working in the organization.

Ex:-

→ select * from Emp Hyd

Union all

select * from Emp Chennai

→ Select * from Emp Hyd

Union

Select * from Emp Chennai

* Write a query to display employee who are working in both branches.

→ Select * from Emp Hyd
Intersect
Select * from Emp Chennai

* Write a query to display employee who are working in Hyderabad but not in Chennai branch.

→ Select * from Emp Hyd
Except
Select * from Emp Chennai

* Write a query to display employee who are working in Chennai but not in hyd branch.

→ Select * from Emp Chennai
Except
Select * from Emp Hyd.

i) Column level alias names :- In this level we are creating alias names for columns in a table.

Syntax :- $\langle \text{ColumnName} \rangle [\text{AS}] \langle \text{Column alias Name} \rangle$

Ex :- DeptNo AS Dept (OR) DeptNo Dept

ii) Table level alias Names :- In this level we are creating alias names on table Name.

Syntax :- $\langle \text{Table Name} \rangle [\text{AS}] \langle \text{Table Alias Name} \rangle$

Ex :- Dept AS D (OR) Dept D

Syntax to Combined Column + table level alias Name :-

Select $\langle \text{Column Name 1} \rangle [\text{AS}] \langle \text{Column Alias Name 1} \rangle, \langle \text{Column Name 2} \rangle [\text{AS}] \langle \text{Column Alias Name 2} \rangle, \dots$ from $\langle \text{TableName} \rangle [\text{AS}] \langle \text{Table Alias Name} \rangle;$

Ex :- select Deptno as x, DName AS y, Loc AS z from Dept AS D.
(OR)

Select DeptNo X, DName y, Loc Z from Dept D.

Ex :-

Create alias
Name

Buffer

Database

table → Dept

DeptNo	DName	Loc
x	x	x
y	y	y
z	z	z

x DeptNo	y DName	z Loc
x	x	x
y	y	y
z	z	z

→ Column Name
replaced with alias
Name.

Display

x	y	z
x	x	x
y	y	y
z	z	z

Note :- Whenever we are creating alias Name's on table or Column internally a database Server is creating the virtual copy on each alias Name & store in buffer memory

Identity (Seed, Increment) :-

It is a

predefined method which is used to generate the identity values on a particular column in the table automatically.

By using identity we will provide autoincrement value facility on table.

- A table should contain only one identity column.

- This method is having the following two arguments are :

i) Seed :- It represent starting value of identity default value is 1.

ii) Increment :- To represent incremental value in between id's, default value is 1.

Identity (Seed, Increment) = identity (1, 1)

~~Ex~~ - Example of identity with default values (seed, increment)

Create table Test1 (SNO Int identity, Name Varchar(30));

Testing :-

Insert Test1 values (1, 'A') -- Error

Insert Test1 values ('A') -- Allowed

Insert Test1 (SNO, Name) values (2, 'B')
 --- Error.

Insert Test1 (Name) values ('B') - Allowed

Table output :-

	SNO	Name
1	1	A
2	2	B

Example of identity with user defined value's

Create table Test2 (SNO Int. identity (100, 5),
 Name Varchar (30))

Testing :-

Insert Test2 values ('A') -- Allowed

Insert Test2 (Name) values ('B') - Allowed

Table output :-

	SNO.	Name
1	100	A
2	105	B

Note :- In the above example's user can not insert value's to an identity column by explicitly.

- If we want to insert value's to an identity column by explicitly then we follow the

following Syntax

Set identity insert <Table Name> off/on

Here,

off :- It is a default Connection of identity, the User can-not insert value's to identity Column by explicitly.

on :- User can insert value's to on identity Column by explicitly.

Ex :- Set identity insert Test on
Insert Test (SNo, Name) values (3, 'C')
--- Allowed.

Ex :- Set identity insert Test off
Insert Test (SNo, Name) values (4, 'D')
--- Error

A	cat	1
d	dog	2

①

Operator's :- To perform some operations on given operands values SQL Server supports the following operators are.

- ① Assignment Operator $\rightarrow =$
- ② Arithmetic operator $\Rightarrow + - * /$
- ③ Relational Operator $\Rightarrow <, >, <=, >=, !<, !>,$
- ④ Logical operator $\rightarrow \text{AND, OR, NOT}$
- ⑤ set operator's $\rightarrow \text{Union, Union all, Intersect, Except}$
- ⑥ Special operator $\rightarrow$

positive operator	Negative operator
In	Not in
between	Not between
is Null	is not null
Like	Not Like

① Assignment Operator :- To assign a value to a variable or to a attribute in SQL

In SQL :-

Syntax :- $\langle \text{Column Name} \rangle \langle \text{Assignment Operator} \rangle \langle \text{value} \rangle$

Ex :- $\text{Select } * \text{ from Emp Where EmpNo} = 7788$

In T/SQL :- Syntax :-

Declare @ $\langle \text{variable Name} \rangle \langle \text{DT} \rangle [\text{Size}]$

Set @ $\langle \text{variable Name} \rangle \langle \text{Assignment operator} \rangle \langle \text{value} \rangle$

Ex:- Declare @ x int
Set @ x = 101

② Arithmetic Operator :- To perform some mathematical calculations like addition, subtraction, multiplication & division

Syntax :- <Column Name> <Arithmetic Operator>
<Value>

Ex:-

① Write a query to display employee salaries after adding 1000

→ Select Salary, Salary + 1000 AS Result
from Emp

② Write a query to display employee Name, job, Salary & Annual Salary of the employee from the table.

Ex:- Select EName, Job, Salary, Salary * 12
AS Annual Salary from Emp.

③ Write a query to update employee salary with a increment of 10% of working with job is analyst.

③

Ex- update Emp Set Salary = Salary +
Salary * 10/100 Where Job = 'Analyst'

④ Write a query to update all employee salaries with a increment of 5%.

Ex- update Emp Set Salary = Salary +
Salary * 0.5

Relational operators :-

Composing a specific Column values with even user defined Conditions.

Syntax :- Where < Column Name > < Relational operator > < Value >

Ex :-

① Write a query to display list of employee who are joined before 1981?

→ Select * from emp where Hiredate < '1981-01-01'

② Write a query to insert studentid, Student Name, and there subject marks find total, average & class of each student

→ perform by your own.

④ logical operators :- To check more than one condition in the query. These operators are AND, OR, NOT

AND Operator :-

Cond1	Cond2	Result
T	T	T
T	F	F
F	T	F
F	F	F

It returns a value when all given conditions are true in that query.

Syntax :- Where <Condition 1> AND <Condition 2> AND <Condition 3> ---

Ex :-

Write a query to display employee who are working in the job is clerk & whose Name is 'Ward'

→ Select * from Emp Where Job = 'clerk' AND Ename = 'Ward'

OR :-

Cond 1	Cond 2	Result
T	T	T
T	F	T
F	T	T
F	F	F

(5)

It returns a value if any one condition is true from the given group of condition

Syntax :-

Where <Condition 1> OR <Condition 2> OR
<Condition 3> OR - - - -

Ex:-

Write a query to display a list of employee who are working under the employee numbers are 7369, 7566, 7788

→ Select * from Emp Where EmpNo = 7369 OR
EmpNo = 7566 OR EmpNo = 7788

Not :- It returns all values except the given Conditional values in the query

Syntax :-

Where NOT <Column Name> = <Value>
AND NOT <Column Name> = <Value> AND - - -

Ex:-

Write a query to delete the list of employee from the table who are ^{not} working under the job is clerk & analyst.

→ Delete from Emp Where NOT JOB = 'clerk'
AND JOB = 'Analyst'

Special operators :- sql server supports the following special operators are

① In operator :- Comparing the group of values based on a single condition in the query.

Syntax :-

Where <column Name> In (<list of values>)

Ex :-

To display the list of employee who are working under the employee numbers are 7369, 7566, 7788.

→ Select * from Emp Where EmpNo In (7369, 7566, 7788)

② NOT In :- It returns the list of values except the given conditional values

Syntax :-

Where <column Name> NOT In (<list of values>)

Ex :- To delete list of employee from the table who are not working under the job is Salesman, Manager, president.

→ Delete from Emp Where JOB NOT In ('Salesman', 'Manager', 'president')

③ Between :- It will work on a particular range values.

Rules -

- ① It returns all values including source & destination values from the given range
- ② It can implement along with AND operator.
- ③ It always use on low values to High values.

Syntax :-

Where < Column Name > Between < low value >
AND < High value >

Ex:- To display employee who's employee salary between 10,000 & 47,000

→ Select * from Emp Salary Where Salary between 10,000 AND 47,000

(OR)

Select * from Emp Where (Salary >= 10,000) AND (Salary <= 47,000)

Between = (>= AND <=)

Ex:- To Display the list of employee who are join in the year of 1981

→ Select * from Emp Where HireDate between '1981-01-01' AND '1981-12-31'

④ NOT between :- It returns all values except the given range value.

Syntax :- Where < Column Name > Not between < low value > AND < High Value >

Ex :- To Display list of employee who are not joined in the year of 1981

→ Select * from Emp Where Hiredate NOT between '1981-01-01' AND '1981-12-31'

⑤ IS Null :-

Comparing Nulls in a table

Syntax :- Where < Column Name > IS Null

Ex :-

Write a query to display the list of employee whose Commission is ~~not~~ Null.

→ Select * from Emp Where Comm IS Null

⑥ IS NOT Null :-

Here we are not Comparing Nulls in a table.

Syntax :- Where < Column Name > IS NOT Null

Ex :- Write a query to display the list of employee whose Commission is not null

→ select * from Emp Where COMM is not Null.

Working with Null :-

- 1) Null is a unknown or undefined value in database.
- 2) Null is not equal to zero & it not equal to space.
- 3) If any arithmetic operator is performing the operation with Null then it again returns Null only.

$$i) a + \text{Null} \rightarrow 1000 + \text{Null} \rightarrow \text{Null}$$

$$ii) a - \text{Null} \rightarrow 1000 - \text{Null} \rightarrow \text{Null}$$

$$iii) a * \text{Null} \rightarrow 1000 * \text{Null} \rightarrow \text{Null}$$

$$iv) a / \text{Null} \rightarrow 1000 / \text{Null} \rightarrow \text{Null}$$

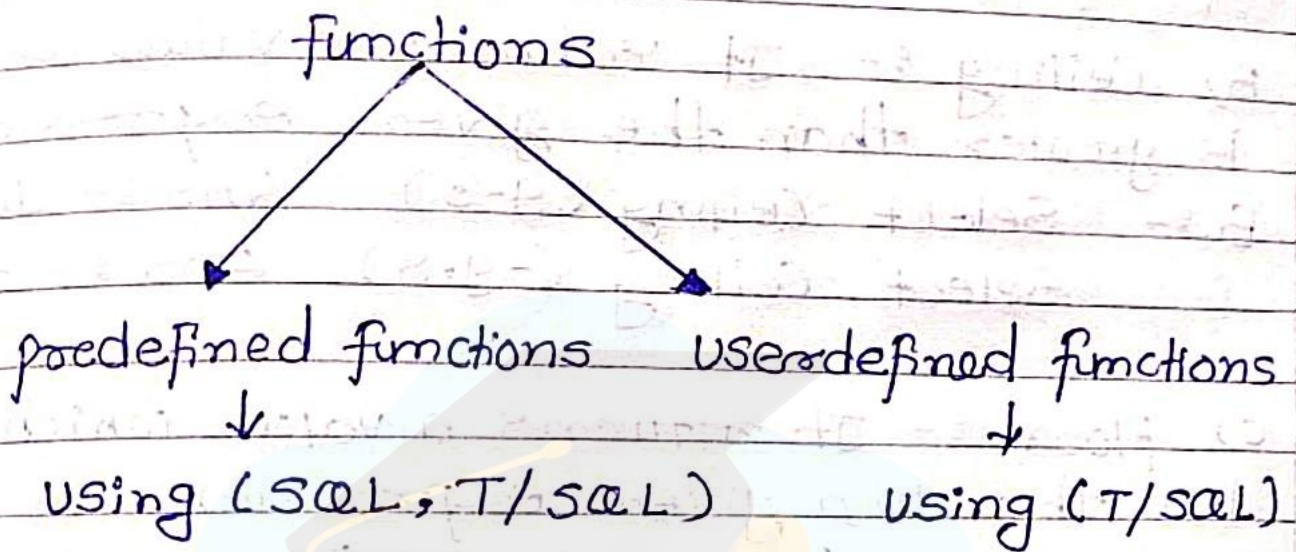
Exs- Write a query to display employee Name, Job, Salary, COMM & also Salary + COMM from the table who's employee Name as Smith.

→ select Ename, job, Salary, COMM, Salary + COMM AS total from Emp Where Ename = 'Smith'

	Ename	Job	Salary	COMM	Total
1	smith	clerk	8000	Null	Null

①
* functions :- A function is nothing but to perform second task as per the given logic & it must returns a value.

- SQL server supports the following two types of functions there are :



predefined functions :- SQL server supports the following predefined function are

- i) Number functions
- ii) Character / String functions
- iii) Date & Time functions
- iv) Aggregative / Grouping functions

Syntax :- `select <Function Name> (value/Expression)`

i) Number function :-

A) ABS () :- It Converts negative sign value into positive sign values

Ex:- Select $ABS(-12)$ AS Result

Ans:- 12

Ex:- Select $ENAME, salary, Comm, ABS$
 $(Comm - Salary)$ AS result from Emp
 AS result from Emp.

B) Ceiling :- It returns a value which is greater than the given expression

Ex:- Select Ceiling $(9-3)$ Ans:- 10

Select Ceiling (-9.8) Ans:- -9

C) Floor :- It returns a value which is less than to a given expression

Ex:- Select floor (9.8) Ans:- 9

Select floor (-9.3) Ans:- -10

d) power :- It returns the power of the given expression

Ex:- select power $(2, 3)$ Ans:- 8

e) pi() :- It returns pi value

Ex:- select pi() Ans = 3.14

f) log() :- It returns the logarithmic value of given expression

Ex:- Select log(3) AS Result Ans:-

1.0986

g) $\log_{10}()$:- It returns base 10 logarithmic value.

Ex:- Select $\log_{10}(10)$ AS result Ans = 1

ii) character or string function :-

a) LEN() function :- It returns the length of given string.

Ex:- Select $\text{LEN}('HELLO')$ Ans :- 5

Ex:- select $\text{LEN}('WELCOME')$ Ans :- 8
 space is also count

Ex:- Select $\text{ENAME}, \text{LEN}(\text{ENAME})$ from Emp

Ex:- Delete from Emp Where $\text{LEN}(\text{ENAME}) > 3$

b) ASCII() :- It returns ascii number for a given character.

Ex:- Select $\text{ASCII}('Z')$ AS Result Ans :- 90

c) CHAR() :- It returns character of the given ASCII No.

Ex:- Select $\text{CHAR}(90)$ AS Result Ans :- Z

d) LOWER() :- It Converts upper case characters into lower case characters.

Ex:- update Emp set $\text{ENAME} = \text{LOWER}(\text{ENAME})$ Where Job = 'clerk'

Ex:- Select $\text{LOWER}('HELLO')$ AS Result

Ans :- hello

e) UPPER() :- It Converts Lower Case characters into upper Case character's
 Ex:- Select UPPER ('hello') AS Result
 Ans :- HELLO

f) LTRIM() :- Trimming the left side space of the given string expression.

Ex:- Select LTRIM ('SAI') AS result
 Ans :- only SAI

g) RTRIM() :- Trimming the ^{right} side space of the given string expression

Ex:- Select RTRIM ('SAI ') AS Result
 Ans:- only SAI

h) REVERSE() :- It reverse the character at the given string

Ex:- select Reverse ('SAI') AS Result
 Ans:- IAS

Ex:- select Ename, Reverse (Ename) from Emp

i) REPLICATE() :- It repeat the given string of character's as per the specified number of times.

Ex:- Select Replicate ('SAI', 5) AS Result

Ans:- SAI, SAI, SAI, SAI, SAI

J) REPLACE() :- To replace an existing characters with new character's in the given string expression

Syntax :- REPLACE ('string', '<old char's>', '<new char's>')

Ex:- select replace ('Jack & Jue', 'J', 'BL') AS Result

Ans:- Black & Blue

K) CONCAT() :- Add two or more than two expression

Ex:- Select Concat ('Good', 'Morning')

AS Result Ans:- GoodMorning
No space

L) SUBSTRING() :- Substring ('string', <starting position of char>, <length of char>)

Ex:- Select Substring ('Welcome', 4, 2)

AS Result

Ans:- CO

⑥

(iii) Date and Time function :-

A) GetDate() :- It returns the current date & time information of the system

Ex:- Select GetDate()

Ans:- Current Date & Time

b) GetUTCDate() :- It returns the current universal date & time information, here UTC stands for Co-ordinate Universal Time.

Ex:- Select GETUTCDATE()

c) Datepart() :- It returns the specified interval from the given date expression.

Ex:- Select Datepart (DD, Getdate())

Ans:- 2 date

Ex:- Select Datepart (HH, GETdate())

Ans:- 20 h's.

d) Datename() :- It returns the name of a specified interval from the given date expression.

Ex:- Select Datename (DD, GETDATE())

Ans:- Friday

Ex:- Select Datename (MM, GETDate())

Ans:- August

e) Dateadd() :- Adding the number of intervals to the given date expression.

Ex:- Select Dateadd (DD, 20, GetDate())

Ex:- Select Dateadd (MM, 10, GetDate());

f) DateDiff() :- It returns the number of intervals in between the given date expressions.

Ex:- Select datediff (DD, '2018-08-02', '2019-08-02')

Ans:- 365

g) Aggregative function :-

(i) Sum() :- It returns the Sum of the given group of values of a Column.

Ex:- Select sum (salary) from Emp

Ans:- Total Salary Ans ex:- 715,000

ii) AVG() :- It returns average of the given group of values of a Column.

Ex:- Select AVG (Salary) from Emp

Ans:- 47666.6666

(iii) MAX() :- It returns MAX value

Ex:- Select MAX (Salary) from Emp

Ans:- 90,000 → Highest Salary

(iv) min() :- It returns MIN value

Ex:- Select MIN (Salary) from Emp

Ans:- 8000 → Lowest Salary

(v) Count() :- This function again classified into 3 types.

(i) Count (*) :- Counting all values including duplicate's and nulls.

Ex:- Select Count (*) from Emp
Emp table

EmpID	EmpName
101	AA
102	BB
103	NULL
104	CC
105	AA
106	CC

Ans:- All values
Counting → 6

(ii) Count (<Column Name>) :-
Counting all values including duplicate's
but NOT NULL'S

⑨

Ex:- Select Count (ENAME) from Emp

Ans :- 5

iii) Count (distinct < Column Name >) :-
Counting unique values from a specific column. Here distinct keyword is used to avoid a counting of duplicate values

Ex:- Select Count (Distinct ENAME) from
Emp

Ans :- 3

